Multi-Vehicle Formation

Multi-Vehicle Formation
1. Course Content
    1.1 Content Description and Formation Design
    1.2 Formation Coordination Implementation Principle
2. Preparation
3. Case Demonstration
4. Core Source Code Analysis

1. Course Content

- This course is based on the **"Leader-Follower"** architecture: the leader vehicle (`robot1`) broadcasts the navigation path and relative-position TF transform globally, while the follower vehicle (`robot2`) resolves the formation goal point it should reach in real time by listening to the coordinates, and follows autonomously. The course presets three formations: "Column", "Row", and "Left/Right Convoy", and supports online switching via dynamic parameters.

1.1 Content Description and Formation Design

The system supports three preset formation layouts (using the center reference frame of the leader vehicle `robot1` as the reference):

Formation Name	Parameter Value (`queue_name`)	Formation Spatial Geometry	Applicable Operation Scenarios
Column	`column`	The follower is located directly behind the leader, arranged in a single-file column: robot2: x = -dist, y = 0	Narrow corridors, single-lane passage
Row	`row`	The follower is distributed directly to the left and right of the leader, in a single horizontal row: `robot2: x = 0, y = -dist`	Large-area floor cleaning, lateral joint inspection

> **Note**
> For teaching convenience, this tutorial focuses on two vehicles (leader `robot1` + follower `robot2`).

1.2 Formation Coordination Implementation Principle

1. **Map and localization reference alignment**: All vehicles access the same LAN and global map service, and establish their own poses under the unified global coordinate system (`map`) through their respective AMCL nodes;

2. **Leader vehicle TF broadcasting**: The leader vehicle (`robot1`) runs the `pub_follower_goal` node. As the leader moves in the map, it continuously broadcasts the expected follower positions (`point2` and `point3`) relative to `robot1/base_link` to the TF tree;
3. **Follower listening and navigation**: The follower vehicle (`robot2`) runs the `get_follower_point` node, which resolves the local goal point `point2` into the global goal pose under the `map` coordinates in real time through TF transform listening, and continuously sends goal points to `/robot2/goal_pose`, driving the follower's Nav2 navigation module to move and follow with obstacle avoidance.

---

2. Preparation

- Referring to the preceding course 【Multi-Vehicle Chassis Control】, set the same `ROS_DOMAIN_ID` and different namespaces for the two vehicles.

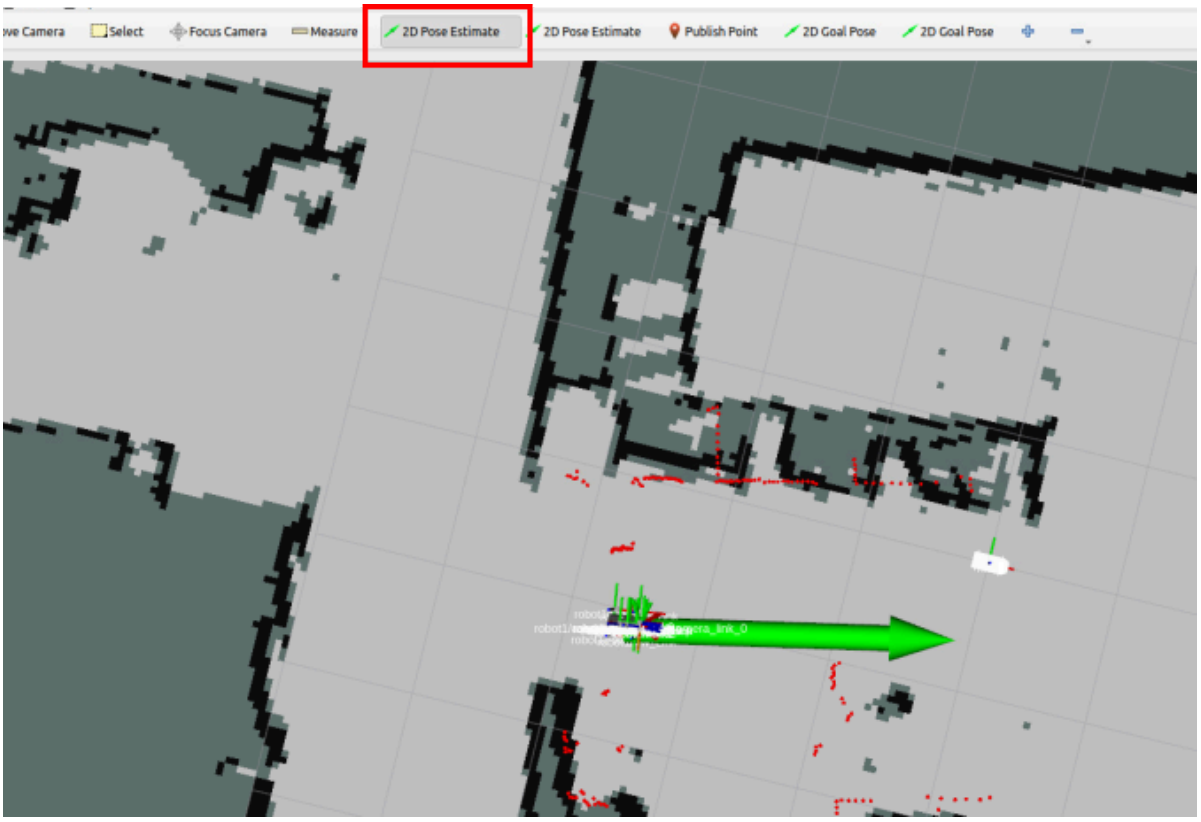
- Start the control program on each of the two vehicles; one vehicle is the primary vehicle and the others are follower vehicles.
- Primary vehicle (robot1) start

```
ros2 launch yahboomcar_bringup multi_robot_demo.launch.py demo:=formation
use_map:=true use_rviz:=true
```

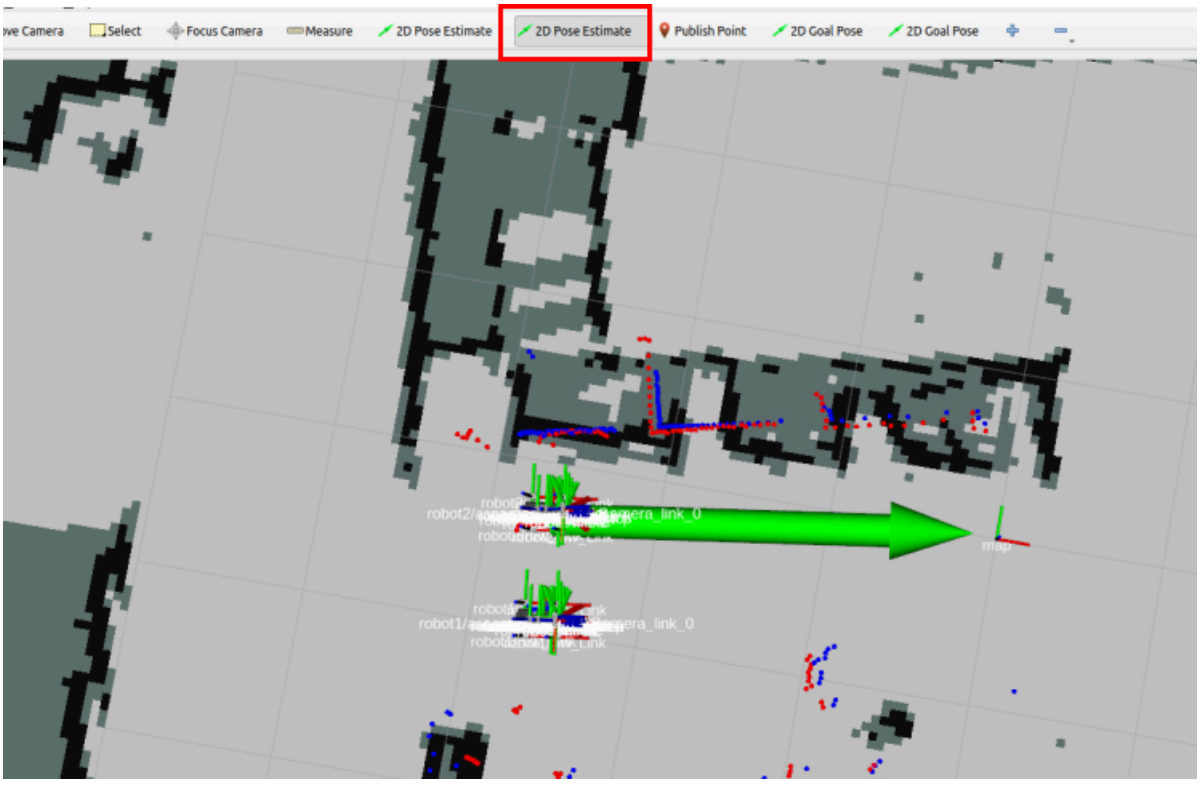
- Follower vehicle (robot2) start

```
ros2 launch yahboomcar_bringup multi_robot_demo.launch.py demo:=formation
use_map:=false
```

- Initialize the robot1 vehicle position: use the first 【2D Pose Estimate】 to initialize the position of vehicle 1; the LiDAR point cloud is red.



- Initialize the robot2 vehicle position: use the second 【2D Pose Estimate】 to initialize the position of vehicle 2; the LiDAR point cloud is blue.



3. Case Demonstration

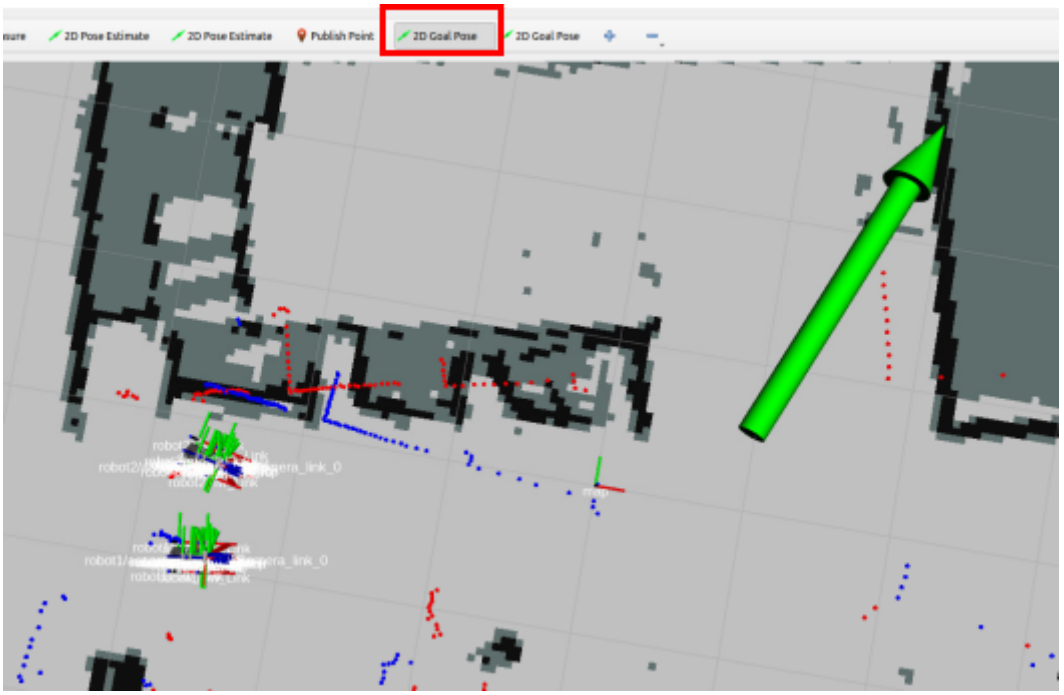
- Switch to row formation

```
ros2 topic pub -r 2 /formation_mode std_msgs/msg/String "{data: row}"
```

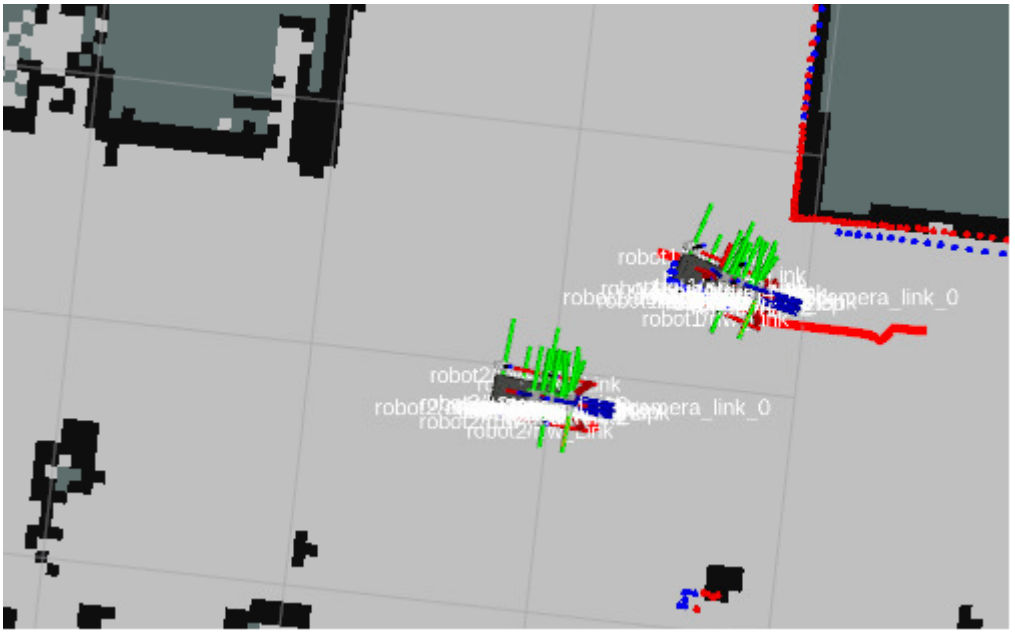
- Switch to column formation

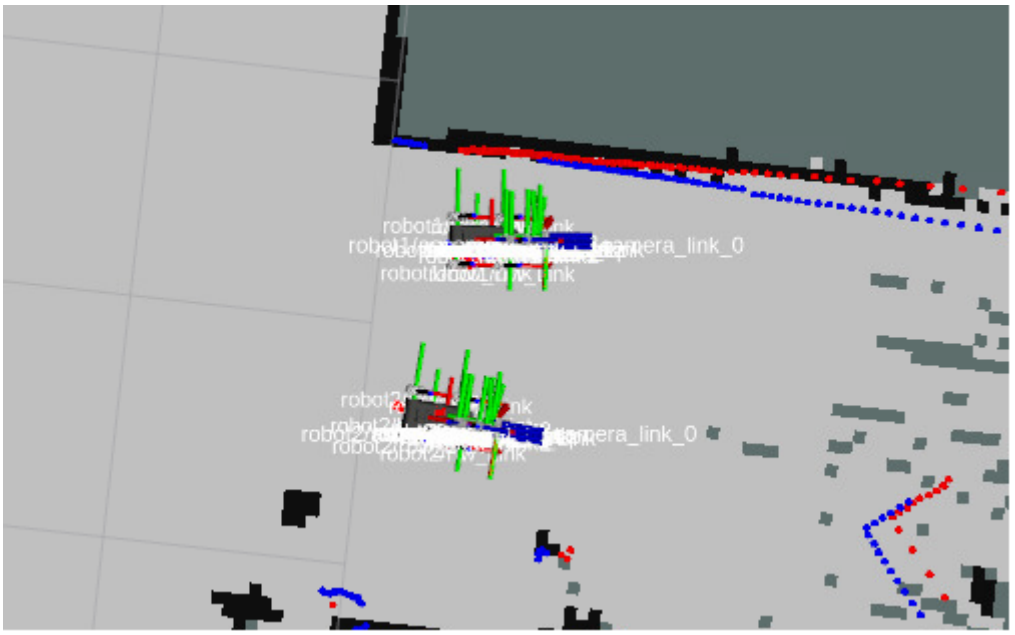
```
ros2 topic pub -r 2 /formation_mode std_msgs/msg/String "{data: column}"
```

- Use the first 【2D Goal Pose】 to send a goal point to `/robot1/goal_pose`. After robot1 starts moving, robot2 will also start moving in the specified formation (row or column), achieving a formation movement effect.

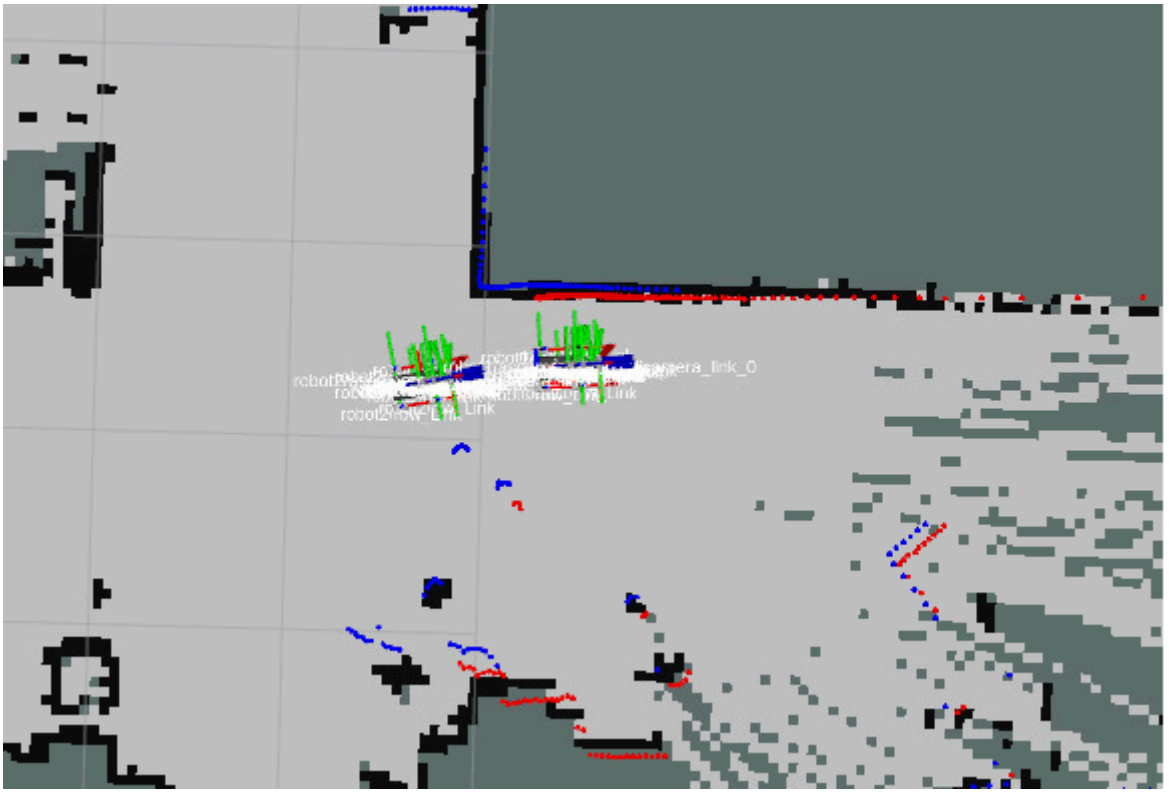


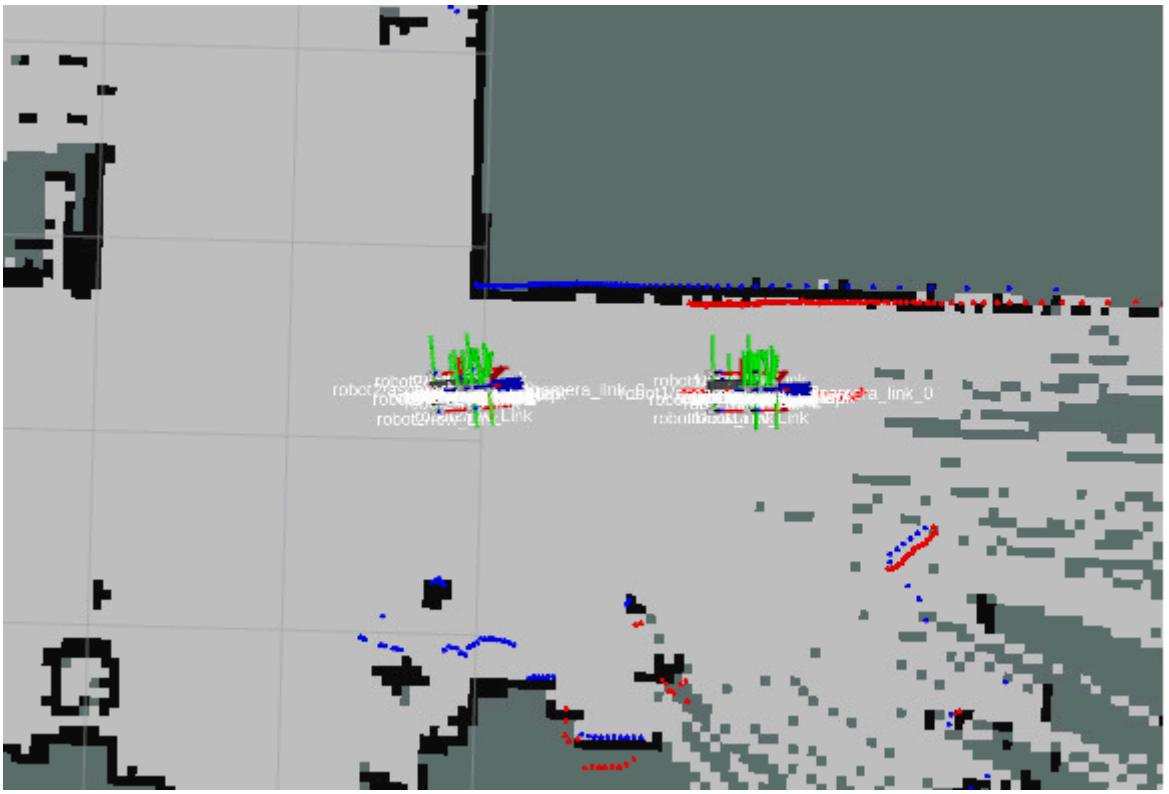
- Row formation movement





- Column formation movement





4. Core Source Code Analysis

- Source code path

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/common_demo/common_demo/multi_robot/multi_robot_formation.py
```

- `multi_robot_formation.py` implements the "Leader-Follower" formation logic: the node subscribes to the leader goal and formation switching topics, periodically queries the leader's real-time pose in `map` through TF, superimposes the formation offset to resolve the follower goal, and then sends it down to the follower's Nav2 navigation stack for autonomous following. The key program fragments are explained module by module below.

Formation constant definition description:

- `FORMATION_ROW = 'row'` / `FORMATION_COLUMN = 'column'`: Defines the string identifiers of the row and column formations.
- `VALID_FORMATIONS`: A tuple of valid formations, used to validate the launch parameter and the formation name input from the `/formation_mode` topic; illegal input triggers a warning and falls back to the default column formation.

`__init__` initialization description:

- `declare_parameter('leader_ns', 'robot1')` / `follower_ns`: Declares the namespaces of the leader vehicle and the follower vehicle, defaulting to `robot1`, `robot2`.
- `formation`: The initial formation, defaulting to column `column`; after reading it is normalized via `.strip().lower()`; if it is not within `VALID_FORMATIONS`, a warning is issued and it falls back to column.
- `spacing`: The formation spacing (unit: m), defaulting to `0.8`, i.e., the offset distance of the follower relative to the leader.

- `update_period`: The timer period (unit: s), defaulting to `0.5`, controlling the refresh frequency of the follower's following goal.
- `position_tolerance` / `yaw_tolerance`: The position/orientation change thresholds; a follower goal is re-sent only when the threshold is exceeded, avoiding frequent goal jitter.
- `tf_timeout`: The TF query timeout (unit: s).
- `_clean_namespace()`: Cleans the namespace string (removing redundant slashes and whitespace).
- `Buffer()` + `TransformListener`: Creates a TF buffer and listener, used to query the leader's real-time pose in the `map` coordinate frame.
- `leader_client` / `follower_client`: The Nav2 `navigate_to_pose` action clients for the two vehicles respectively, each attached under the corresponding namespace.
- Subscribes to `/{leader_ns}/goal_pose`: Receives the leader goal published by the RViz 【2D Goal Pose】.
- Subscribes to `/formation_mode`: Receives runtime formation switching commands (`row` / `column`).
- State variables: `activated` (starts following only after the first leader goal is received), `pending_leader_goal` (caches the goal while the leader navigation stack is not ready), `last_follower_goal` (the last successfully sent follower goal), `last_warn_times` (warning throttling), `last_goal_log_time` (log throttling).
- `create_timer(update_period, timer_callback)`: Creates a periodic timer to continuously refresh the follower's following goal.

Callback function description:

- `leader_goal_callback`: After receiving a new leader goal, rebuilds a `PoseStamped` (frame defaults to `map`, updating the timestamp); on the first trigger, sets `activated=True` to start following; calls `send_nav_goal` to forward the goal to the leader navigation; on success clears the cache, and on failure stores it in `pending_leader_goal` and issues a throttled warning; finally calls `update_follower_goal(force=True)` to refresh the follower immediately according to the new goal.
- `formation_mode_callback`: Switches formation at runtime; first normalizes and validates the input, returning with a warning if illegal; ignores it if it is the same as the current formation; otherwise updates `self.formation`, and if already activated, clears `last_follower_goal` and forcibly refreshes the follower goal.
- `timer_callback`: The periodic callback. If a cached leader goal exists, it tries to resend it (in case the navigation stack becomes ready late); if not activated, it returns directly; otherwise it refreshes the follower goal with `force=False` (sending down only when the change exceeds the threshold).

Formation resolution and goal dispatch description:

- `update_follower_goal(force)`: First calls `compute_follower_goal` to resolve the follower goal; if the resolution fails, it returns; when not forced and the goal change does not exceed the threshold, it skips; on dispatch failure, it issues a throttled warning and returns; on success, it records `last_follower_goal` and prints a log according to `force` or a 2-second throttle.

- `compute_follower_goal`: The core formation resolution. Queries `map -> {leader_ns}/base_footprint` through TF to obtain the leader's global pose `(x, y, yaw)`; determines the offset in the leader base frame according to the formation — row `offset=(0, -spacing)` (the follower is on the right of the leader), column `offset=(-spacing, 0)` (the follower is directly behind the leader); then uses the 2D rotation formula to transform the local offset into the global `map` coordinate frame; the follower's heading stays consistent with the leader's (converted from `yaw` to a quaternion).
- `goal_changed`: Determines whether the displacement `hypot(dx, dy)` between the new goal and the last sent goal exceeds `position_tolerance`, or whether the orientation difference exceeds `yaw_tolerance`, used for dispatch debouncing.
- `send_nav_goal`: Sends a `NavigateToPose` goal to the specified Nav2 action client; if the server is not ready (`server_is_ready()` is False), returns False, otherwise asynchronously calls `send_goal_async` and returns True.

```
FORMATION_ROW = 'row'
FORMATION_COLUMN = 'column'
VALID_FORMATIONS = (FORMATION_ROW, FORMATION_COLUMN)


class MultiRobotFormation(Node):

    def __init__(self):
        super().__init__('multi_robot_formation')

        # ---- Parameters ----
        self.declare_parameter('leader_ns', 'robot1')
        self.declare_parameter('follower_ns', 'robot2')
        self.declare_parameter('formation', FORMATION_COLUMN)
        self.declare_parameter('spacing', 0.8)
        self.declare_parameter('update_period', 0.5)
        self.declare_parameter('position_tolerance', 0.15)
        self.declare_parameter('yaw_tolerance', 0.15)
        self.declare_parameter('tf_timeout', 0.1)

        self.leader_ns =
self._clean_namespace(self.get_parameter('leader_ns').value)
        self.follower_ns =
self._clean_namespace(self.get_parameter('follower_ns').value)
        formation = str(self.get_parameter('formation').value).strip().lower()
        if formation not in VALID_FORMATIONS:
            self.get_logger().warn(
                f'unknown formation "{formation}", fallback to "
{FORMATION_COLUMN}"')
            formation = FORMATION_COLUMN
        self.formation = formation
        self.spacing = float(self.get_parameter('spacing').value)
        update_period = float(self.get_parameter('update_period').value)
        self.position_tolerance =
float(self.get_parameter('position_tolerance').value)
        self.yaw_tolerance = float(self.get_parameter('yaw_tolerance').value)
        self.tf_timeout = float(self.get_parameter('tf_timeout').value)

        # ---- TF: query the current leader pose in the map frame ----
```

```
        self.tf_buffer = Buffer()
        self.tf_listener = TransformListener(self.tf_buffer, self)

        # ---- Navigation action clients: each robot's nav2 runs under its own
namespace ----
        self.leader_client = ActionClient(
            self, NavigateToPose, f'/{self.leader_ns}/navigate_to_pose')
        self.follower_client = ActionClient(
            self, NavigateToPose, f'/{self.follower_ns}/navigate_to_pose')

        # ---- Leader goal (rviz 2D Goal Pose) and runtime formation switching --
--
        self.create_subscription(
            PoseStamped, f'/{self.leader_ns}/goal_pose',
self.leader_goal_callback, 1)
        self.create_subscription(
            String, '/formation_mode', self.formation_mode_callback, 1)

        # ---- State ----
        self.activated = False            # follower tracking starts only after a
leader goal is received
        self.pending_leader_goal = None   # goal cached while the leader nav
stack is not ready
        self.last_follower_goal = None    # most recent follower goal
successfully sent
        self.last_warn_times = {}
        self.last_goal_log_time = 0.0

        self.create_timer(update_period, self.timer_callback)
        self.get_logger().info(
            f'multi_robot_formation started: leader={self.leader_ns}, '
            f'follower={self.follower_ns}, formation={self.formation}, '
            f'spacing={self.spacing:.2f}m')

    # ------------------------------------------------------------------ #
    # Callbacks
    # ------------------------------------------------------------------ #
    def leader_goal_callback(self, msg):
        """A new leader goal arrived: forward it to the leader navigation and
activate/update follower tracking"""
        goal = PoseStamped()
        goal.header.frame_id = msg.header.frame_id or 'map'
        goal.header.stamp = self.get_clock().now().to_msg()
        goal.pose = msg.pose

        if not self.activated:
            self.activated = True
            self.last_follower_goal = None
            self.get_logger().info('leader goal received, follower tracking
activated')

        if self.send_nav_goal(self.leader_client, goal):
            self.pending_leader_goal = None
            self.get_logger().info(
                f'leader goal: x={goal.pose.position.x:.2f}, '
                f'y={goal.pose.position.y:.2f}')
```

```
        else:
            self.pending_leader_goal = goal
            self._warn_throttled(
                'leader_server',
                f'{self.leader_ns} navigate_to_pose not available, will retry')

        # Refresh the follower goal immediately based on the new target
        self.update_follower_goal(force=True)

    def formation_mode_callback(self, msg):
        """Switch formation at runtime: row / column"""
        mode = str(msg.data).strip().lower()
        if mode not in VALID_FORMATIONS:
            self.get_logger().warn(
                f'invalid formation_mode "{msg.data}", expected row/column')
            return
        if mode == self.formation:
            return
        self.formation = mode
        self.get_logger().info(f'formation switched to {mode}')
        if self.activated:
            self.last_follower_goal = None
            self.update_follower_goal(force=True)

    def timer_callback(self):
        # Re-send the cached goal when the leader nav stack becomes ready late
        if self.pending_leader_goal is not None:
            if self.send_nav_goal(self.leader_client, self.pending_leader_goal):
                self.get_logger().info('leader goal re-sent after server ready')
                self.pending_leader_goal = None
        if not self.activated:
            return
        self.update_follower_goal(force=False)

    # ------------------------------------------------------------------ #
    # Formation computation and goal dispatch
    # ------------------------------------------------------------------ #
    def update_follower_goal(self, force=False):
        """Compute the follower goal for the current formation; send it when the
change exceeds the threshold"""
        goal = self.compute_follower_goal()
        if goal is None:
            return
        if not force and self.last_follower_goal is not None \
                and not self.goal_changed(goal):
            return
        if not self.send_nav_goal(self.follower_client, goal):
            self._warn_throttled(
                'follower_server',
                f'{self.follower_ns} navigate_to_pose not available '
                f'(goal x={goal.pose.position.x:.2f}, y=
{goal.pose.position.y:.2f}), '
                f'keep waiting')
            return
        self.last_follower_goal = goal
        now = self.get_clock().now().nanoseconds / 1e9
```

```
        if force or now - self.last_goal_log_time > 2.0:
            self.get_logger().info(
                f'follower goal ({self.formation}): '
                f'x={goal.pose.position.x:.2f}, y={goal.pose.position.y:.2f}')
            self.last_goal_log_time = now

    def compute_follower_goal(self):
        """Follower goal = leader current pose plus the formation offset (defined
in the leader base frame)"""
        leader_frame = f'{self.leader_ns}/base_footprint'
        try:
            transform = self.tf_buffer.lookup_transform(
                'map', leader_frame, rclpy.time.Time(),
                timeout=rclpy.duration.Duration(seconds=self.tf_timeout))
        except TransformException as e:
            self._warn_throttled('tf', f'wait TF map -> {leader_frame}: {e}')
            return None

        x = transform.transform.translation.x
        y = transform.transform.translation.y
        yaw = self.quaternion_to_yaw(transform.transform.rotation)

        if self.formation == FORMATION_ROW:
            # Row formation: the follower is on the right side of the leader
(negative y direction)
            offset_x, offset_y = 0.0, -self.spacing
        else:
            # Column formation: the follower is directly behind the leader
(negative x direction)
            offset_x, offset_y = -self.spacing, 0.0

        goal = PoseStamped()
        goal.header.frame_id = 'map'
        goal.header.stamp = self.get_clock().now().to_msg()
        goal.pose.position.x = x + offset_x * math.cos(yaw) - offset_y *
math.sin(yaw)
        goal.pose.position.y = y + offset_x * math.sin(yaw) + offset_y *
math.cos(yaw)
        # The follower keeps the same heading as the leader while moving in
formation
        goal.pose.orientation.z = math.sin(yaw * 0.5)
        goal.pose.orientation.w = math.cos(yaw * 0.5)
        return goal

    def goal_changed(self, goal):
        """Whether the position/orientation difference between the new goal and
the last sent goal exceeds the threshold"""
        dx = goal.pose.position.x - self.last_follower_goal.pose.position.x
        dy = goal.pose.position.y - self.last_follower_goal.pose.position.y
        if math.hypot(dx, dy) > self.position_tolerance:
            return True
        yaw = self.quaternion_to_yaw(goal.pose.orientation)
        last_yaw =
self.quaternion_to_yaw(self.last_follower_goal.pose.orientation)
        return abs(self.normalize_angle(yaw - last_yaw)) > self.yaw_tolerance
```

```
    def send_nav_goal(self, client, pose):
        """Send a NavigateToPose goal to the given nav2; return False if the
server is not ready"""
        if not client.server_is_ready():
            return False
        goal = NavigateToPose.Goal()
        goal.pose = pose
        client.send_goal_async(goal)
        return True
```